

Prompt: Planning — Color Check-in hoy + Bloqueo Early/Late

Contexto del código existente

- `PlanningCellStatus` está en `shared/schema.ts` como tipo literal union
 - `getStatusColor()` y `getStatusLabel()` están en el frontend (planning page)
 - El loop de occupancy está en `server/storage.ts` → `getPlanningData()`
 - Las celdas con reserva usan `getSourceColor(reservation.source)` (color por origen)
 - Las celdas sin reserva usan `getStatusColor(status)` (color por estado)
 - El `reservationsMap` ya incluye `earlyCheckIn`, `earlyCheckInTime`, `lateCheckOut`, `lateCheckOutTime`
-

CAMBIO 1 — Color diferenciado para check-in del día y ocupadas

Problema

Las reservas con check-in hoy se muestran con el mismo color que cualquier otra reserva futura (color por origen). No hay forma visual de distinguir “entra hoy” vs “ya está adentro”.

Paso 1A — Agregar dos nuevos status al tipo en `shared/schema.ts`

Buscar:

```
export type PlanningCellStatus = "available" | "booked" | "checked_in" | "checkou
```

Reemplazar por:

```
export type PlanningCellStatus = "available" | "booked" | "checkin_today" | "chec
```

Paso 1B — Asignar `checkin_today` en el backend (`server/storage.ts`)

En `getPlanningData()` , dentro del loop donde se asigna la occupancy, buscar el bloque:

```
if (reservation) {
  cellReservations[room.id][day] = reservation.id;
  const isGroupRes = groupReservationIds.has(reservation.id);
  if (isGroupRes) {
    occupancy[room.id].push("group_blocked");
  } else if (reservation.status === "checked_in") {
    occupancy[room.id].push("checked_in");
  } else {
    occupancy[room.id].push("booked");
  }
}
```

Reemplazar por:

```
if (reservation) {
  cellReservations[room.id][day] = reservation.id;
  const isGroupRes = groupReservationIds.has(reservation.id);
  if (isGroupRes) {
    occupancy[room.id].push("group_blocked");
  } else if (reservation.status === "checked_in") {
    if (day === reservation.checkOutDate) {
      occupancy[room.id].push("checkout_today");
    } else {
      occupancy[room.id].push("checked_in");
    }
  } else if (day === reservation.checkInDate && day === todayStr) {
    // Reserva confirmada con check-in hoy
    occupancy[room.id].push("checkin_today");
  } else {
    occupancy[room.id].push("booked");
  }
}
```

Nota: aplicar el mismo cambio en el segundo `getPlanningData` que existe para el storage en memoria (in-memory), que tiene la misma estructura.

Paso 1C — Colores y labels en el frontend (planning page)

En `getStatusColor()` , agregar el nuevo case (antes del `default`):

```
case "checkin_today":
```

```

    return "bg-teal-100 dark:bg-teal-900/40 border-teal-300 dark:border-teal-700";
case "early_blocked":
    return "bg-orange-50 dark:bg-orange-900/20 border-orange-200 dark:border-orange";
case "late_blocked":
    return "bg-purple-50 dark:bg-purple-900/20 border-purple-200 dark:border-purple";

```

En `getStatusLabel()` , agregar:

```

case "checkin_today":
    return "Check-in hoy";
case "early_blocked":
    return "Bloqueado (Early check-in)";
case "late_blocked":
    return "Bloqueado (Late check-out)";

```

Paso 1D — Las reservas con `checkin_today` usan color de estado, no de origen

Actualmente todas las celdas con reserva usan `getSourceColor(reservation.source)` . Las de check-in hoy deben destacarse con su color propio. En el JSX de la celda, buscar:

```

className={`h-8 rounded border flex items-center justify-center transition-all cu
    getSourceColor(reservation.source)
  } hover:ring-2 hover:ring-primary/50`}

```

Reemplazar por:

```

className={`h-8 rounded border flex items-center justify-center transition-all cu
    status === "checkin_today"
      ? getStatusColor("checkin_today")
      : getSourceColor(reservation.source)
  } hover:ring-2 hover:ring-primary/50`}

```

Paso 1E — Agregar a la leyenda del planning

En el array `statusItems` de la leyenda, agregar:

```

{ status: "checkin_today", label: "Check-in hoy" },

```

Ponerlo después de `{ status: "available", label: "Disponible" }` .

CAMBIO 2 — Early check-in y late check-out bloquean celdas adyacentes

Lógica

- Si reserva tiene `earlyCheckIn = true` → el día **anterior** al check-in queda bloqueado como `early_blocked` (no se puede crear reserva, se muestra ícono Sunrise)
- Si reserva tiene `lateCheckOut = true` → el día **del check-out** (que actualmente es “libre” porque las reservas van hasta `checkOut - 1`) queda bloqueado como `late_blocked` (no se puede crear reserva, se muestra ícono Sunset)

Esto es correcto: en el planning la reserva ocupa `checkIn` hasta `checkOut - 1`. El día `checkOut` queda libre. Con late check-out ese día debe bloquearse.

Paso 2A — En el backend, marcar celdas bloqueadas por early/late

En `getPlanningData()`, después del loop principal de occupancy (después del `for (const day of days)` que ya existe), agregar un segundo pasaje que detecta early/late y marca los días adyacentes.

Buscar el cierre del loop `for (const day of days)` dentro del `for (const room of allRooms)` y agregar justo después (antes del cierre del `for (const room of allRooms)`):

```
// Segundo pasaje: marcar días bloqueados por early check-in / late check-out
for (let dayIndex = 0; dayIndex < days.length; dayIndex++) {
  const day = days[dayIndex];

  // Ya está marcado con algo — no sobrescribir
  const currentStatus = occupancy[room.id][dayIndex];
  if (currentStatus !== "available") continue;

  // Buscar si alguna reserva de esta habitación tiene early check-in en el día s
  const nextDay = days[dayIndex + 1];
  if (nextDay) {
    const earlyRes = allReservations.find(r =>
      r.roomId === room.id &&
      r.checkInDate === nextDay &&
      (r.earlyCheckIn === true || r.earlyCheckIn === "true")
    );
    if (earlyRes) {
      occupancy[room.id][dayIndex] = "early_blocked";
      // Guardar referencia a la reserva para mostrar el ícono
      cellReservations[room.id][day] = earlyRes.id;
    }
  }
}
```

```

        continue;
    }
}

// Buscar si alguna reserva de esta habitación tiene late check-out HOY (checkO
const lateRes = allReservations.find(r =>
    r.roomId === room.id &&
    r.checkOutDate === day &&
    (r.lateCheckOut === true || r.lateCheckOut === "true")
);
if (lateRes) {
    occupancy[room.id][dayIndex] = "late_blocked";
    // Guardar referencia a la reserva para mostrar el ícono y datos
    cellReservations[room.id][day] = lateRes.id;
}
}

```

Nota: aplicar el mismo cambio en el `getPlanningData` del storage en memoria.

Paso 2B — En el frontend, renderizar las celdas `early_blocked` y `late_blocked`

Actualmente el JSX tiene dos ramas:

1. `{reservation ? <DraggableReservationCell> : <div clickeable/no-clickeable>}`

Hay que agregar una tercera rama para `early_blocked` y `late_blocked`: celdas no arrastrables, no clickeables para crear, que muestran solo el ícono.

Reemplazar el bloque de renderizado de cada celda por:

```

{reservation && status !== "early_blocked" && status !== "late_blocked" ? (
    // Celda con reserva normal — igual que antes
    <DraggableReservationCell
        id={`drag-${reservationId}-${room.id}-${day}`}
        reservationId={reservationId!}
        roomId={room.id}
        onClick={() => handleCellClick(room, day, status, reservationId)}
        className={`h-8 rounded border flex items-center justify-center transition-all
            status === "checkin_today"
                ? getStatusColor("checkin_today")
                : getSourceColor(reservation.source)
            } hover:ring-2 hover:ring-primary/50`}
        data-testid={`cell-${room.id}-${day}`}
    >
        <span className="text-[10px] font-medium truncate px-1 max-w-[56px] inline-fl

```

```

        {reservation.earlyCheckIn && day === reservation.checkIn && (
            <Sunrise className="h-3 w-3 text-orange-400 flex-shrink-0" data-testid="i
        )}
        {reservation.isGroup ? "GRP" : reservation.guestName.split(" ")[0]}
        {reservation.lateCheckOut && (() => {
            const coDate = new Date(reservation.checkOut + "T12:00:00");
            coDate.setDate(coDate.getDate() - 1);
            const lastDay = coDate.toISOString().split("T")[0];
            return day === lastDay;
        })() && (
            <Sunset className="h-3 w-3 text-purple-400 flex-shrink-0" data-testid="ic
        )}
    </span>
</DraggableReservationCell>
) : status === "early_blocked" ? (
    // Celda bloqueada por early check-in - solo muestra ícono, no permite crear
    <div
        className={`h-8 rounded border flex items-center justify-center ${getStatusCo
        data-testid={`cell-${room.id}-${day}`}
    >
        <Sunrise className="h-3 w-3 text-orange-400" />
    </div>
) : status === "late_blocked" ? (
    // Celda bloqueada por late check-out - solo muestra ícono, no permite crear
    <div
        className={`h-8 rounded border flex items-center justify-center ${getStatusCo
        data-testid={`cell-${room.id}-${day}`}
    >
        <Sunset className="h-3 w-3 text-purple-400" />
    </div>
) : (
    // Celda libre o con estado sin reserva (maintenance, dirty, etc.)
    <div
        onClick={() => handleCellClick(room, day, status, reservationId)}
        className={`h-8 rounded border flex items-center justify-center transition-al
        getStatusColor(status)
    } ${
        isClickable
            ? "cursor-pointer hover:ring-2 hover:ring-primary/50 hover:scale-105"
            : "cursor-default"
        }`
        data-testid={`cell-${room.id}-${day}`}
    >
        {isClickable ? (
            <Plus className="h-3 w-3 text-green-600 dark:text-green-400 opacity-0 group
        ) : null}
    </div>

```

```
}}
```

Paso 2C — Actualizar `handleCellClick` para que los status bloqueados no abran el dialog de nueva reserva

En `handleCellClick`, agregar los nuevos status a la condición que no permite crear:

```
const handleCellClick = (room: RoomWithType, day: string, status: PlanningCellSta
// No abrir nada en celdas bloqueadas por early/late
if (status === "early_blocked" || status === "late_blocked") return;

if (status === "available") {
  setSelectedCell({ ... });
  setQuickReservationOpen(true);
} else {
  const resolvedId = reservationId || findReservationForRoomAndDay(room.id, day
  if (resolvedId) {
    setSelectedReservationId(resolvedId);
    setReservationDetailOpen(true);
  }
}
};
```

Paso 2D — Tooltip explicativo para celdas early/late bloqueadas

En el `TooltipContent` de cada celda, el tooltip ya muestra el estado via

`getStatusLabel(status)`. Con los labels agregados en el Paso 1C, el tooltip mostrará automáticamente "Bloqueado (Early check-in)" o "Bloqueado (Late check-out)".

Para que el tooltip también muestre a qué reserva corresponde el bloqueo, en la sección del tooltip que renderiza `{reservation && ...}`, el bloqueo sí tiene una referencia en `cellReservations[room.id][day]` (se la guardamos en el Paso 2A), así que `reservation` no será null y el tooltip mostrará el nombre del huésped con la razón del bloqueo.

Paso 2E — Agregar a la leyenda

En el array `statusItems` agregar al final:

```
{ status: "early_blocked", label: "Early check-in" },
{ status: "late_blocked", label: "Late check-out" },
```

Resumen visual

Estado	Color	Ícono	Clickeable
<code>checkin_today</code>	Teal/verde azulado	—	Sí (abre detalle)
<code>checked_in</code>	Amber/amarillo	—	Sí (abre detalle)
<code>booked</code>	Azul (por origen)	—	Sí (abre detalle)
<code>early_blocked</code>	Naranja tenue + borde punteado	 Sunrise	No (bloqueado)
<code>late_blocked</code>	Violeta tenue + borde punteado	 Sunset	No (bloqueado)

Archivos a modificar

Archivo	Cambios
<code>shared/schema.ts</code>	Ampliar tipo <code>PlanningCellStatus</code>
<code>server/storage.ts</code>	Loop de occupancy: agregar <code>checkin_today</code> ; segundo pasaje: early/late blocked (×2: DB y memoria)
<code>client/src/pages/planning.tsx</code>	<code>getStatusColor</code> , <code>getStatusLabel</code> , JSX de celda, <code>handleCellClick</code> , leyenda